files

CSCI 134

previously on csci 134

each team member returned to woodle hq with a list of the woods they'd discovered


```
member1 = ["BALSA", "BEECH", "EBONY"]
```

```
member2 = ["ASPEN", "BIRCH", "CEDAR", "EBONY", "MAPLE"]
```

```
member3 = ["BEECH", "BIRCH", "CEDAR", "MAPLE"]
```

previously on csci 134

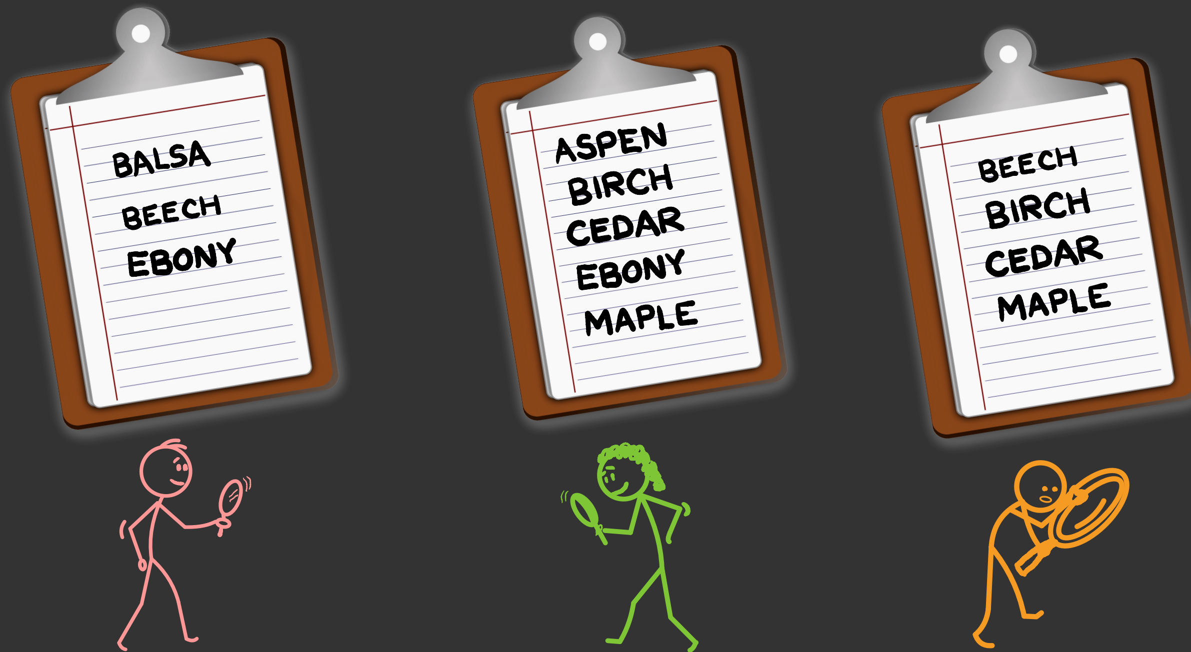
each team member returned to woodle hq with a list of the woods they'd discovered

```
member1 = ["BALSA", "BEECH", "EBONY"]
```

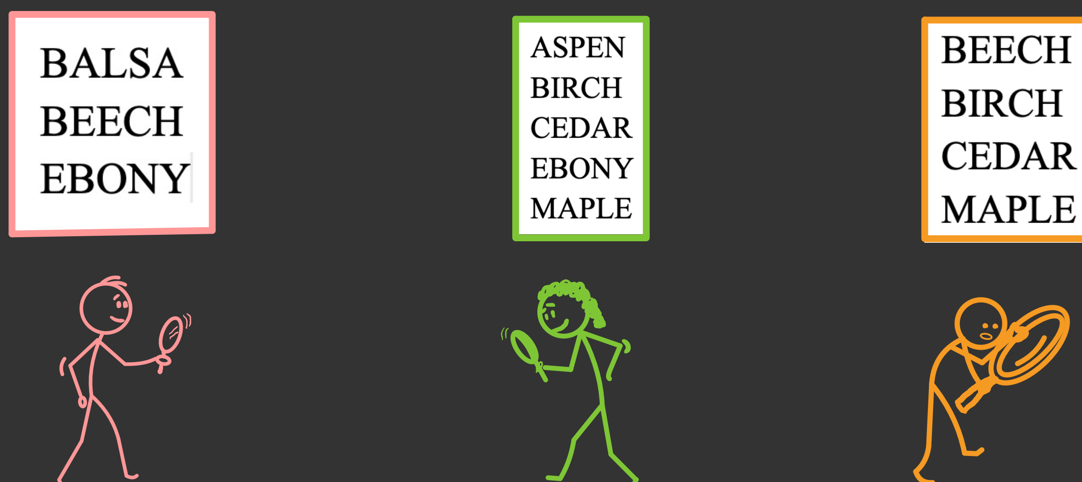
```
member2 = ["ASPEN", "BIRCH", "CEDAR", "EBONY", "MAPLE"]
```

```
member3 = ["BEECH", "BIRCH", "CEDAR", "MAPLE"]
```

(but that's not exactly how it happened) actually the researchers came back with field notes



they typed them into microsoft word



so we ended up with three text files

member1.txt

BALSA
BEECH
EBONY

member2.txt

ASPEN
BIRCH
CEDAR
EBONY
MAPLE

member3.txt

BEECH
BIRCH
CEDAR
MAPLE

how do we convert these text files to python lists?

member1.txt

```
BALSA
BEECH
EBONY
```

→

```python
member1 = ["BALSA", "BEECH", "EBONY"]
```

member2.txt

```
ASPEN
BIRCH
CEDAR
EBONY
MAPLE
```

→

```python
member2 = ["ASPEN", "BIRCH", "CEDAR", "EBONY", "MAPLE"]
```

member3.txt

```
BEECH
BIRCH
CEDAR
MAPLE
```

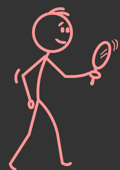
→

```python
member3 = ["BEECH", "BIRCH", "CEDAR", "MAPLE"]
```

sorta like this

```
woods = []
with open('member1.txt') as reader:
    for line in reader:
        woods = woods + [line]
```

but why the **blank lines**?



member1.txt

```
BALSA
BEECH
EBONY
```

```
In [1]: with open('member1.txt') as reader:
   ...:     for line in reader:
   ...:         print(line)
   ...:
BALSA

BEECH

EBONY
```

member2.txt

```
ASPEN
BIRCH
CEDAR
EBONY
MAPLE
```

```
In [2]: with open('member2.txt') as reader:
   ...:     for line in reader:
   ...:         print(line)
   ...:
ASPEN

BIRCH

CEDAR

EBONY

MAPLE
```

and what are **these**?

member1.txt

```
BALSA
BEECH
EBONY
```

```
In [1]: woods = []

In [2]: with open('member1.txt') as reader:
   ...:     for line in reader:
   ...:         woods = woods + [line]
   ...:

In [3]: woods
Out[3]: ['BALSA\n', 'BEECH\n', 'EBONY\n']
```

when we store text files, there is no such thing as **lines**

Ozymandias

BY PERCY BYSSHE SHELLEY

I met a traveller from an antique land,
Who said—"Two vast and trunkless legs of stone
Stand in the desert. . . . Near them, on the sand,
Half sunk a shattered visage lies, whose frown,
And wrinkled lip, and sneer of cold command,
Tell that its sculptor well those passions read
Which yet survive, stamped on these lifeless things,
The hand that mocked them, and the heart that fed;
And on the pedestal, these words appear:
My name is Ozymandias, King of Kings;
Look on my Works, ye Mighty, and despair!
Nothing beside remains. Round the decay
Of that colossal Wreck, boundless and bare
The lone and level sands stretch far away."

imagine a sequence of lights that encodes a message

it's just one long sequence, so we'll need a **special symbol** to encode "moving to a new line"

this "newline symbol" is written \n

member1.txt

```
BALSA
BEECH
EBONY
```

```
In [1]: woods = []

In [2]: with open('member1.txt') as reader:
   ...:     for line in reader:
   ...:         woods = woods + [line]
   ...:

In [3]: woods
Out[3]: ['BALSA\n', 'BEECH\n', 'EBONY']
```

a newline character has length 1

```
In [1]: len("BALSA\n")
Out[1]: 6

In [2]: len("FIR\n")
Out[2]: 4

In [3]: len("\n")
Out[3]: 1
```

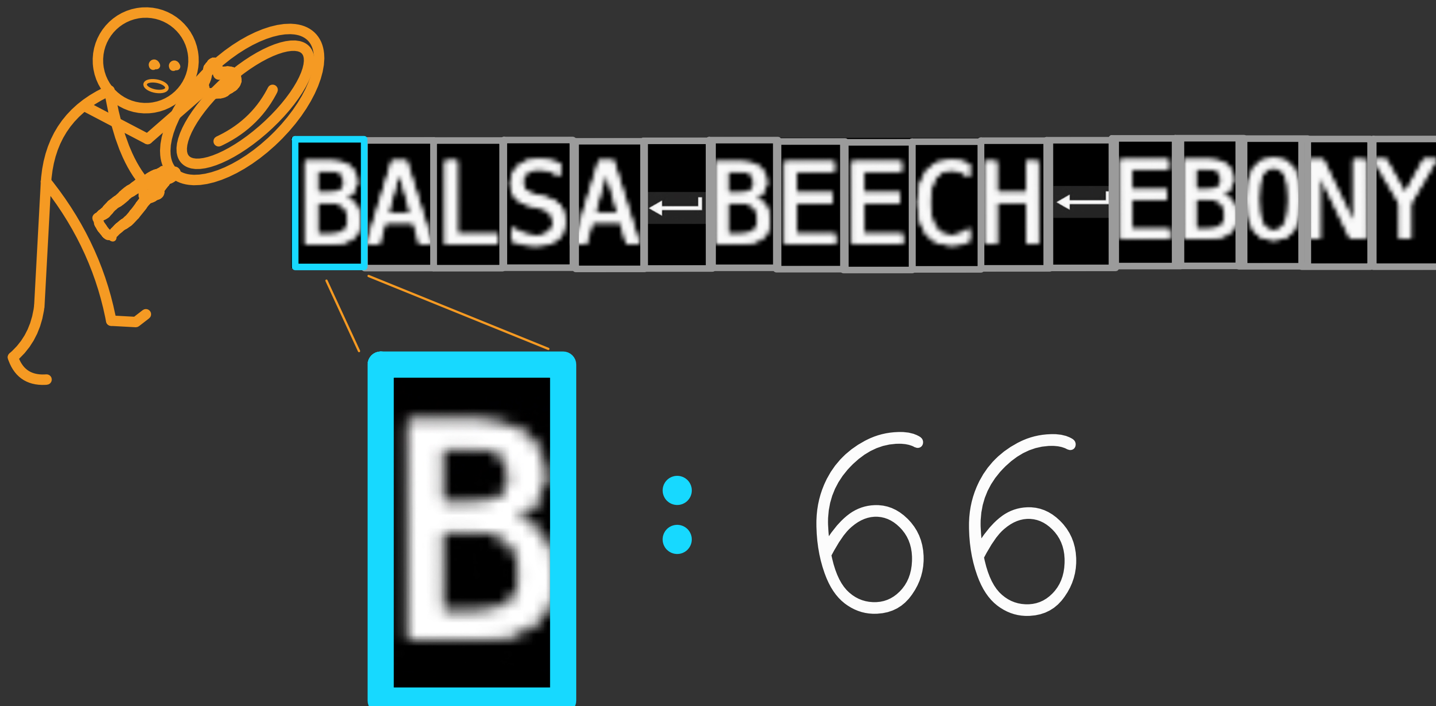
if you print a string containing newline characters, it will display them as new lines

```
In [1]: print("BALSA\nMAPLE\nCEDAR")
BALSA
MAPLE
CEDAR

In [2]: print("CSCI\n134")
CSCI
134
```

but how is each character stored?

each character is stored as
a **predetermined number**



each character is stored as a predetermined number

B	A	L	S	A	↵	B	E	E	C	H	↵	E	B	O	N	Y
66	65	76	83	65	10	66	69	69	67	72	10	69	66	79	78	89

we can ask for these numbers using the **ord** function

```
In [1]: ord("B")
Out[1]: 66

In [2]: ord("C")
Out[2]: 67

In [3]: ord("\n")
Out[3]: 10
```

we can ask for the character using the **chr** function

```
In [1]: chr(66)
Out[1]: 'B'

In [2]: chr(67)
Out[2]: 'C'

In [3]: chr(10)
Out[3]: '\n'
```

there are many available characters

```
In [1]: chr(128300)
Out[1]: '🔬'

In [2]: chr(128301)
Out[2]: '🔭'

In [3]: chr(128302)
Out[3]: '🔮'
```

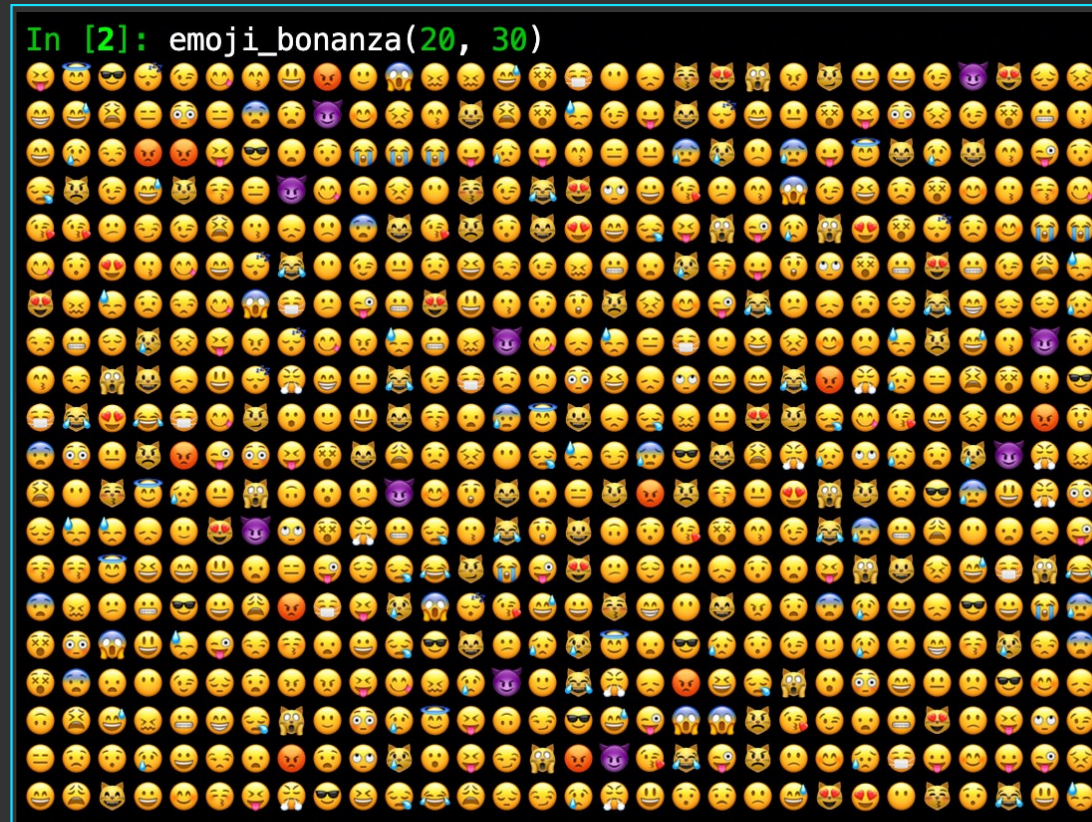
← look at small things

← look at distant things

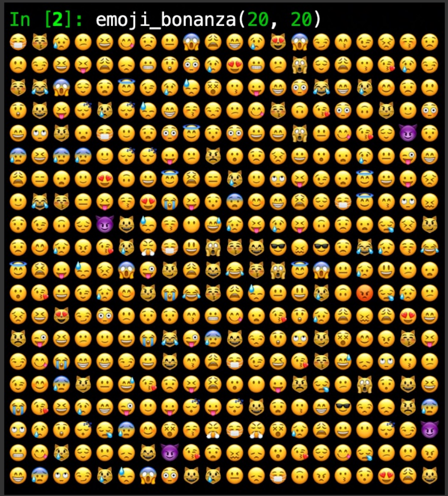
← look at future things

this allows us to do

great things



this allows us to do great things




```python
from random import randint

def emoji_bonanza(num_lines, chars_per_line):
    for line_index in range(num_lines):
        next_line = ""
        for char_index in range(chars_per_line):
            # 128512 to 128580 is a bunch of face emojis
            emoji = chr(randint(128512, 128580))
            next_line = next_line + emoji
        print(next_line)
```

the number associated with a character is called its unicode encoding

```
In [1]: ord("👽")
Out[1]: 128125

In [2]: ord("👀")
Out[2]: 128064

In [3]: ord("h")
Out[3]: 104
```

the unicode encodings of uppercase and lowercase letters are 32 apart

65	A	+ 32 =	97	a
66	B	+ 32 =	98	b
67	C	+ 32 =	99	c
68	D	+ 32 =	100	d
69	E	+ 32 =	101	e

```python
def lowercase(line):
    """Converts the line to lowercase."""
    result = ""
    for character in line:
        unicode = ord(character)
        if unicode >= 65 and unicode <= 65+26:
            next_char = chr(unicode + 32)
        else:
            next_char = character
        result = result + next_char
    return result

def trim(line):
    """Trims the newline from the end of a line."""
    if len(line) > 0 and line[-1] == "\n":
        return line[:-1]
    else:
        return line

def read_and_lowercase(filename):
    """Reads in a file and lowercases each line."""
    with open(filename) as reader:
        for line in reader:
            line = trim(line)
            print(lowercase(line))
```

65	A	+ 32 =		97	a
66	B	+ 32 =		98	b
67	C	+ 32 =		99	c
68	D	+ 32 =		100	d
69	E	+ 32 =		101	e

```
In [3]: read_and_lowercase('ozymandias.txt')
ozymandias

i met a traveller from an antique land,
who said—“two vast and trunkless legs of stone
stand in the desert.... near them, on the sand,
half sunk a shattered visage lies, whose frown,
and wrinkled lip, and sneer of cold command,
tell that its sculptor well those passions read
which yet survive, stamped on these lifeless things,
the hand that mocked them, and the heart that fed;
and on the pedestal, these words appear:
my name is ozymandias, king of kings;
look on my works, ye mighty, and despair!
nothing beside remains. round the decay
of that colossal wreck, boundless and bare
the lone and level sands stretch far away.”
```

in the wheel of fortune bonus round,
contestants are given the letters

RSTLNE

in the wheel of fortune bonus round, contestants are given the letters

RSTLNE

are these really the five most common consonants?

is this really the most common vowel?

strategy

read in a text file and count the letters using a dictionary

```
def get_lines(filename):
    """Reads the lines from a file."""
    lines = []
    with open(filename) as reader:
        for line in reader:
            lines = lines + [line]
    return lines

def letter_frequency(filename):
    """Counts the frequency of lower case letters."""
    lines = get_lines(filename)
    alphabet = 'abcdefghijklmnopqrstuvwxyz'
    frequencies = dict()
    for letter in alphabet:
        frequencies[letter] = 0
    for line in lines:
        for character in line:
            if character in alphabet:
                frequencies[character] += 1
    return frequencies

if __name__ == "__main__":
    frequencies = letter_frequency("mars.txt")
    for letter in frequencies:
        frequency = frequencies[letter]
        print(letter + ": " + str(frequency))
```

read a text file

```python
def get_lines(filename):
    """Reads the lines from a file."""
    lines = []
    with open(filename) as reader:
        for line in reader:
            lines = lines + [line]
    return lines

def letter_frequency(filename):
    """Counts the frequency of lower case letters."""
    lines = get_lines(filename)
    alphabet = 'abcdefghijklmnopqrstuvwxyz'
    frequencies = dict()
    for letter in alphabet:
        frequencies[letter] = 0
    for line in lines:
        for character in line:
            if character in alphabet:
                frequencies[character] += 1
    return frequencies

if __name__ == "__main__":
    frequencies = letter_frequency("mars.txt")
    for letter in frequencies:
        frequency = frequencies[letter]
        print(letter + ": " + str(frequency))
```

initialize the **dictionary**

```
{'a': 0,
 'b': 0,
 'c': 0,
 'd': 0,
 'e': 0,
 'f': 0,
 'g': 0,
 'h': 0,
 'i': 0,
 'j': 0,
 'k': 0,
 'l': 0,
 'm': 0,
 'n': 0,
 'o': 0,
 'p': 0,
 'q': 0,
 'r': 0,
 's': 0,
 't': 0,
 'u': 0,
 'v': 0,
 'w': 0,
 'x': 0,
 'y': 0,
 'z': 0}
```

```python
def get_lines(filename):
    """Reads the lines from a file."""
    lines = []
    with open(filename) as reader:
        for line in reader:
            lines = lines + [line]
    return lines

def letter_frequency(filename):
    """Counts the frequency of lower case letters."""
    lines = get_lines(filename)
    alphabet = 'abcdefghijklmnopqrstuvwxyz'
    frequencies = dict()
    for letter in alphabet:
        frequencies[letter] = 0
    for line in lines:
        for character in line:
            if character in alphabet:
                frequencies[character] += 1
    return frequencies

if __name__ == "__main__":
    frequencies = letter_frequency("mars.txt")
    for letter in frequencies:
        frequency = frequencies[letter]
        print(letter + ": " + str(frequency))
```

count the lowercase letters

```
{'a': 340,
 'b': 77,
 'c': 116,
 'd': 150,
 'e': 562,
 'f': 103,
 'g': 128,
 'h': 279,
 'i': 335,
 'j': 7,
 'k': 62,
 'l': 181,
 'm': 104,
 'n': 318,
 'o': 322,
 'p': 69,
 'q': 2,
 'r': 233,
 's': 295,
 't': 394,
 'u': 87,
 'v': 33,
 'w': 111,
 'x': 2,
 'y': 61,
 'z': 5}
```

```python
def get_lines(filename):
    """Reads the lines from a file."""
    lines = []
    with open(filename) as reader:
        for line in reader:
            lines = lines + [line]
    return lines

def letter_frequency(filename):
    """Counts the frequency of lower case letters."""
    lines = get_lines(filename)
    alphabet = 'abcdefghijklmnopqrstuvwxyz'
    frequencies = dict()
    for letter in alphabet:
        frequencies[letter] = 0
    for line in lines:
        for character in line:
            if character in alphabet:
                frequencies[character] += 1
    return frequencies

if __name__ == "__main__":
    frequencies = letter_frequency("mars.txt")
    for letter in frequencies:
        frequency = frequencies[letter]
        print(letter + ": " + str(frequency))
```

print the letter frequencies

```
% python freq.py
a: 340
b: 77
c: 116
d: 149
e: 563
f: 102
g: 127
h: 279
i: 335
j: 7
k: 62
l: 180
m: 104
n: 319
o: 321
p: 69
q: 2
r: 233
s: 293
t: 392
u: 86
v: 33
w: 111
x: 2
y: 60
z: 5
```

```python
def get_lines(filename):
    """Reads the lines from a file."""
    lines = []
    with open(filename) as reader:
        for line in reader:
            lines = lines + [line]
    return lines

def letter_frequency(filename):
    """Counts the frequency of lower case letters."""
    lines = get_lines(filename)
    alphabet = 'abcdefghijklmnopqrstuvwxyz'
    frequencies = dict()
    for letter in alphabet:
        frequencies[letter] = 0
    for line in lines:
        for character in line:
            if character in alphabet:
                frequencies[character] += 1
    return frequencies

if __name__ == "__main__":
    frequencies = letter_frequency("mars.txt")
    for letter in frequencies:
        frequency = frequencies[letter]
        print(letter + ": " + str(frequency))
```


```
% python freq.py
a: 340
b: 77
c: 116
d: 149
e: 563
f: 102
g: 127
h: 279
i: 335
j: 7
k: 62
l: 180
m: 104
n: 319
o: 321
p: 69
q: 2
r: 233
s: 293
t: 392
u: 86
v: 33
w: 111
x: 2
y: 60
z: 5
```

most common vowel

5 out of 6 most common consonants


```
% python freq.py
a: 340
b: 77
c: 116
d: 149
e: 563
f: 102
g: 127
h: 279
i: 335
j: 7
k: 62
l: 180
m: 104
n: 319
o: 321
p: 69
q: 2
r: 233
s: 293
t: 392
u: 86
v: 33
w: 111
x: 2
y: 60
z: 5
```

most common **vowel**

5 out of 6 most common **consonants**

woe is me